Primzahltest und Berechenbarkeit

Jana Chung
jana.chung@uni-bielefeld.de

8. April 2026

Die vorliegende Ausarbeitung orientiert sich in Aufbau und Inhalt maßgeblich an dem Lehrbuch [1].

1 Was ist ein Algorithmus

Ein Algorithmus ist eine endliche, wohldefinierte Folge von Anweisungen, die für jede zulässige Eingabe nach endlich vielen Schritten terminiert und eine Ausgabe liefert. Relevant sind folgende Aspekte:

- Welche Eingaben akzeptiert er?
- Welche Rechenschritte werden in welcher Reihenfolge durchgeführt?
- Wann stoppt der Algorithmus? Was wird dann ausgegeben?

Algorithmen können von Menschen ausgeführt werden, in Hardwares implementiert werden oder in Software (d.h. als Computerprogramm in der Programmiersprache) geschrieben werden.

2 Berechnungsprobleme

Durch eine Funktion $f\colon A \to B$ können wir ausdrücken, dass ein Computerprogramm bei Eingabe $a \in A$ das Ergebnis $f(a) \in B$ ausgibt. Da Computer meist intern nur mit Nullen und Einsen rechnen, erlauben wir als Ein- und Ausgabe andere Zeichenketten.

Definition 1. *Sei A eine nichtleere endliche Menge. Für $k \in \mathbb{N} \cup \{0\}$ bezeichnen wir mit A^k die Menge der Funktionen $f : 1, \ldots, k] \to A$. Ein Element $f \in A^k$ heißt **Wort / Zeichenkette** der **Länge** k über dem **Alphabet** A. Wir schreiben es als Folge $f(1)...f(k)$. Das einzige Element $\emptyset$ von A^0 nennen wir das **leere Wort** (mit Länge 0). Wir setzen $A^* := \bigcup_{k \in \mathbb{N} \cup \{0\}} A^k$. Eine **Sprache** über dem Alphabet A ist eine Teilmenge von A^*.*

Auf dieser Definition baut die folgende auf:

Definition 2. *Ein* ***Berechnungsproblem*** *ist eine Relation* $P \subseteq D \times E$*, wobei es zu jedem* $d \in D$ *mindestens ein* $e \in E$ *gibt mit* $(d, e) \in P$*. Wenn* $(d, e) \in P$*, dann ist* e *eine* ***korrekte Ausgabe*** *für das Problem* P *mit Eingabe* d*. Die Elemente von* D *heißen* ***Instanzen*** *des Problems.* P *heißt* ***eindeutig****, wenn* P *eine Funktion ist. Sind* D*,* E *Sprachen über einem endlichen Alphabet* A*, so sprechen wir von einem* ***diskreten Berechnungsproblem****. Sind* D *und* E *Teilmengen von* $\mathbb{R}^m$ *bzw.* $\mathbb{R}^n$ *für* m*,* $n \in \mathbb{N}$*, so sprechen wir von einem* ***numerischen Berechnungsproblem****. Ein eindeutiges Berechnungsproblem* $P\colon D \to E$ *mit* $|E| = 2$ *heißt* ***Entscheidungsproblem****.*

3 Pseudocode

Algorithmen sollen Berechnungsprobleme lösen. Der Computer, der für die Ausführung am praktischsten ist, benötigt einen Maschinencode, Dieser ist für Menschen schwer lesbar. Deshalb nutzt man Programmiersprachen, die verständlich sind und mittels Compilers in einen Maschinencode übersetzt werden können. Es passieren also folgende Schritte:

Algorithmus entwerfen
→ wird in Computerprogramm (z.B. in C++) implementiert
→ wird zu Maschinencode kompiliert
→ Ausführung

Eine übliche Möglichkeit, Algorithmen zu konkretisieren, sind sogenannten **Pseudocodes** - dieser ist für Menschen noch sehr verständlich und relativ leicht in eine Programmiersprache umzusetzen. Hier wird besonders die Idee des Algorithmus deutlich.

4 Primzahltest

4.1 Idee des Algorithmus

Im Folgenden untersuchen wir, ob eine gegebene natürliche Zahl prim ist oder nicht. Dafür definieren wir zuerst, was überhaupt prim bedeutet.

Definition 3. *Eine natürliche Zahl* n *heißt prim, wenn* $n \geq 2$ *ist und es keine natürliche Zahl* a *und* b *gibt mit* $a > 1$*,* $b > 1$ *und* $n = a \cdot b$

Das Entscheidungsproblem hier lautet
$\{(n, e) \in \mathbb{N} \times \{0, 1\} | e = 1 \iff \text{n prim}\}$.

Eingabe: $n \in \mathbb{N}$.
Frage: Ist n prim?

Dieses Problem soll mit einem Algorithmus gelöst werden, dem sogenannte Primzahltest. Diesen formulieren wir in einem Pseudocode. Zuvor erklären wir Code-Anweisungen und führen noch einige Definition ein.

If-Anweisung: Führt Anweisungen abhängig vom Wahrheitswert einer Bedingung aus. Ist die Bedingung wahr, wird der `then`-Teil ausgeführt, andernfalls der `else`-Teil (optional).

For-Anweisung: Eine Variable durchläuft sukzessive Werte, und für jeden Wert wird ein Anweisungsblock ausgeführt.

Definition 4 (Gaußklammern)**.** *Für $x \in \mathbb{R}$ definieren wir:*

$$\lfloor x \rfloor := \max\{k \in \mathbb{Z} \mid k \leq x\}, \quad \lceil x \rceil := \min\{k \in \mathbb{Z} \mid k \geq x\}.$$

Definition 5 (Modulo-Operation)**.** *Für $a, b \in \mathbb{N}$ definieren wir:*

$$a \bmod b := a - b \cdot \left\lfloor \frac{a}{b} \right\rfloor.$$

Es gilt:

$$b \mid a \iff a \bmod b = 0.$$

Algorithm 1 Einfacher Primzahltest

Require: $n \in \mathbb{N}$
Ensure: Antwort auf die Frage, ob n prim ist

```
if n = 1 then
    antwort ← "nein"
else
    antwort ← "ja"
    for i ← 2 to ⌊√n⌋ do
        if i | n then
            antwort ← "nein"
return antwort
```

Anstelle von $\sqrt{n}$ kann man die Schleifenbedingung auch als $i \cdot i \leq n$ formulieren, um Wurzelberechnungen zu vermeiden.

Der Algorithmus terminiert für alle $n \in \mathbb{N}$, d. h. er berechnet die Funktion

$$f(n) = \begin{cases} \text{ja}, & \text{falls } n \text{ prim ist}, \\ \text{nein}, & \text{sonst}. \end{cases}$$

5 Sieb des Eratosthenes

Im nächsten Abschnitt betrachten wir ein Beispiel für ein allgemeineres diskretes Berechnungsproblem: **das Sieb des Eratosthenes**. Nach Eingabe einer natürlichen Zahl n besteht die Aufgabe darin, alle Primzahlen p mit p $\leq$ n zu berechnen. Der Algorithmus lautet wie folgt:

Satz 6. *Obiger Algorithmus ist korrekt.*

Algorithm 2 Sieb des Eratosthenes

Require: $n \in \mathbb{N}$
Ensure: alle Primzahlen, die nicht größer als n sind

```
for i ← 2 to n do p[i] ← "ja"
for i ← 2 to n do
    if p[i] = "ja" then
        return i
        for j ← i to ⌊n/i⌋ do
            p[i · j] ← "nein".
```

Proof. Wir zeigen die Korrektheit des Algorithmus. Dazu beweisen wir für alle $k \in \{2, \ldots, n\}$:

$$k \text{ wird genau dann ausgegeben, wenn } k \text{ prim ist.}$$

Dies zeigen wir durch das Beweisen in beide Richtungen.

(1) Ist k prim, so wird k ausgegeben.
Sei $k \in \{2, \ldots, n\}$ eine Primzahl.

Im Algorithmus wird eine Zahl k genau dann nicht ausgegeben, wenn irgendwann $p[k] \leftarrow$ "nein" gesetzt wird. Wir zeigen, dass dies für Primzahlen nicht passiert.

Die Zuweisung $p[k] \leftarrow$ "nein" erfolgt nur dann, wenn es Zahlen $i, j \in \mathbb{N}$ mit $i \geq 2$, $j \geq i$ gibt, sodass

$$k = i \cdot j.$$

Das bedeutet aber, dass k als Produkt zweier Zahlen größer oder gleich 2 dargestellt werden kann. Dies widerspricht der Annahme, dass k prim ist.

Also wird für eine Primzahl k niemals $p[k] =$ "nein" gesetzt. Somit gilt während des gesamten Algorithmus:

$$p[k] = \text{“ja”}.$$

Folglich wird k ausgegeben.

(2) Ist k nicht prim, so wird k nicht ausgegeben.
Sei nun $k \in \{2, \ldots, n\}$ keine Primzahl. Dann ist k zusammengesetzt, also existieren Zahlen $i, j \in \mathbb{N}$ mit

$$k = i \cdot j \quad \text{und} \quad 2 \leq i \leq j.$$

Wir wählen i als den kleinsten Teiler von k, der mindestens 2 ist. Dann gilt insbesondere:

$$2 \leq i \leq j = \frac{k}{i}.$$

Da i der kleinste Teiler von k ist, kann i selbst keine nichttrivialen Teiler besitzen. Also ist i eine Primzahl.

Nach Teil (1) wissen wir, dass für Primzahlen stets $p[i] =$ "ja" gilt, wenn der Algorithmus die Zahl i betrachtet.

In diesem Schritt führt der Algorithmus die innere Schleife aus und setzt für alle geeigneten j:

$$p[i \cdot j] \leftarrow \text{“nein”}.$$

Da $k = i \cdot j$, wird insbesondere auch

$$p[k] \leftarrow \text{“nein”}$$

gesetzt.

Ab diesem Zeitpunkt bleibt $p[k] =$ "nein", da dieser Wert im Algorithmus nicht mehr geändert wird.

Wenn später im äußeren Durchlauf der Wert $i = k$ erreicht wird, gilt daher:

$$p[k] = \text{“nein”},$$

und somit wird k nicht ausgegeben.

Damit ist die Korrektheit des Algorithmus gezeigt. □

6 Nicht alles ist berechenbar: Das Halteproblem

Das Halteproblem ist das Beispiel dafür, dass man mit Algorithmen nicht alles berechnen kann. Zunächst wollen wir zeigen, dass alle Sprachen, also auch Programme, abzählbar sind. Dafür benötigen wir:

Lemma 7. *Seien $k, l \in \mathbb{N}$. Definiere die Abbildung*

$$f^l : \{0, \ldots, k-1\}^l \to \{0, \ldots, k^l - 1\}, \quad f^l(w) := \sum_{i=1}^{l} a_i k^{l-i}.$$

für alle $w = a_1, \ldots, a_l \in \{0, ..., k-1\}^l$. Dann ist die Abbildung f^l wohldefiniert und bijektiv.

Die Grundidee des Lemmas lässt sich mit diesem Beispiel veranschaulichen. Wenn wir die Zahl 257 betrachten, verstehen wir darunter die Summe $2 \cdot 10^2 + 5 \cdot 10^1 + 7 \cdot 10^0$. Genau das macht die Abbildung f^l für jede Basis k. In dem gerade betrachteten Beispiel, also unseren normalen Zählsystem rechnen wir mit der Basis 10 und haben Ziffern von 0 bis 9.

Machen wir ein genaueres Beispiel: $k = 2, l = 3$

Die möglichen Tupel sind:

$(0,0,0), (0,0,1), (0,1,0), (0,1,1), (1,0,0), (1,0,1), (1,1,0), (1,1,1)$.

Wendet man nun f^3 an, so ergibt sich zum Beispiel für das konkrete Tupel (0,1,1) der Wert $0 \cdot 4 + 1 \cdot 2 + 1 \cdot 1 = 3$. Jedem Tupel wird also eine Zahl, von 0 bis 7 zugewiesen.

Die Bijektivität bedeutet also, dass eine Zahl zwischen 0 und $k^l - 1$ dasselbe ist wie eine Folge von l Ziffern zur Basis k.

Dieses Lemma wollen wir nun beweisen.

Proof. **1. Wohldefiniertheit:**

Sei $(a_1, \ldots, a_l) \in \{0, \ldots, k-1\}^l$. Dann gilt für alle i:

$$0 \leq a_i \leq k-1.$$

Damit folgt

$$0 \leq a_i k^{l-i} \leq (k-1)k^{l-i}.$$

Durch Aufsummieren erhalten wir

$$0 \leq f_l(a_1, \ldots, a_l) = \sum_{i=1}^{l} a_i k^{l-i} \leq \sum_{i=1}^{l} (k-1)k^{l-i}.$$

Die rechte Summe ist eine geometrische Reihe:

$$\sum_{i=1}^{l} (k-1)k^{l-i} = (k-1)\sum_{i=1}^{l} k^{l-i} = (k-1)\sum_{i=0}^{l-1} k^{l-i-1}$$

$$= (k-1)\sum_{j=0}^{l-1} k^j = (k-1) \cdot \frac{k^l - 1}{k-1} = k^l - 1.$$

Also gilt insgesamt

$$0 \leq f_l(a_1, \ldots, a_l) \leq k^l - 1,$$

und damit ist f_l wohldefiniert.

2. Injektivität:

Seien $(a_1, \ldots, a_l), (b_1, \ldots, b_l) \in \{0, \ldots, k-1\}^l$ mit

$$(a_1, \ldots, a_l) \neq (b_1, \ldots, b_l)$$

beliebig aber fest gewählt. Dann gibt es einen kleinsten Index $j \in \{1, \ldots, l\}$ mit

$$a_j \neq b_j.$$

Ohne Einschränkung sei $a_j > b_j$.

Wir betrachten die Differenz:

$$f^l(a_1, \ldots, a_l) - f^l(b_1, \ldots, b_l) = \sum_{i=1}^{l} (a_i - b_i)k^{l-i}.$$

Für $i < j$ gilt $a_i = b_i$, also verschwinden diese Terme:

$$= (a_j - b_j)k^{l-j} + \sum_{i=j+1}^{l} (a_i - b_i)k^{l-i}.$$

Nun schätzen wir die Summe für $i > j$ nach unten ab. Da $a_i, b_i \in \{0, \ldots, k-1\}$ gilt

$$a_i - b_i \geq -(k-1).$$

Also

$$\sum_{i=j+1}^{l} (a_i - b_i)k^{l-i} \geq -\sum_{i=j+1}^{l} (k-1)k^{l-i}.$$

Wie oben ergibt die geometrische Summe:

$$\sum_{i=j+1}^{l} (k-1)k^{l-i} = k^{l-j} - 1.$$

Damit folgt

$$\sum_{i=j+1}^{l} (a_i - b_i)k^{l-i} \geq -(k^{l-j} - 1).$$

Zusammen ergibt sich:

$$f^l(a_1, \ldots, a_l) - f^l(b_1, \ldots, b_l) \geq (a_j - b_j)k^{l-j} - (k^{l-j} - 1).$$

Da $a_j > b_j$, gilt $a_j - b_j \geq 1$, also:

$$\geq k^{l-j} - (k^{l-j} - 1) = 1.$$

Insbesondere ist

$$f^l(a_1, \ldots, a_l) > f^l(b_1, \ldots, b_l).$$

Also ist f^l injektiv.

3. Surjektivität:
Die Mengen haben dieselbe Mächtigkeit:

$$|\{0, \ldots, k-1\}^l| = k^l = |\{0, \ldots, k^l - 1\}|.$$

Die linke Menge besteht aus allen Tupeln der Länge l. Für jede Position gibt es k verschiedene Möglichkeiten. Die Anzahl ergibt sich also als Produkt $k \cdot k \cdot k \ldots k = k^l$. Die rechte Menge besteht einfach aus allen Zahlen von 0 bis $k^l - 1$, weist also auch die Anzahl k^l auf (am Ende zieht man zwar 1 ab, fängt aber mit 0 an).

Da f^l eine injektive Abbildung zwischen zwei endlichen Mengen gleicher Größe ist, ist sie automatisch auch surjektiv.

f^l ist somit bijektiv. □

Satz 8. *Sei A eine nichtleere Menge. Dann ist die Menge A^* abzählbar.*

Der Beweis wird in Folge nur skizziert:

Beweisskizze. Die Idee des Beweises besteht darin, die Wörter aus A^* systematisch zu nummerieren.

Zunächst werden alle Wörter nach ihrer Länge geordnet. Für jedes $l \in \mathbb{N}$ gibt es genau $|A|^l$ Wörter der Länge l. Da A endlich ist, ist somit auch jede Menge A^l endlich.

Anschließend werden die Wörter fester Länge l eindeutig als Zahlen im $|A|$-adischen Zahlensystem kodiert. Dazu wird jedem Symbol aus A eine Ziffer aus $\{0, \ldots, |A| - 1\}$ zugeordnet.

Durch diese Kodierung erhält man eine injektive Abbildung von A^* nach $\mathbb{N}$, da Wörter unterschiedlicher Länge in verschiedene Bereiche fallen und die Darstellung innerhalb fester Länge eindeutig ist.

Damit ist A^* abzählbar. □

Korollar 9. *Die Menge aller C++-Programme ist abzählbar.*

Proof. C++-Programme sind Wörter über einem endlichen Alphabet. Die zu zeigende Aussage folgt also aus Satz 9. □

Satz 10. *Es gibt Funktionen $f : \mathbb{N} \to \{0, 1\}$, die von keinem C++-Programm berechnet werden.*

Proof. Sei $\mathcal{P}$ die nach Korrolar 9 abzählbare Menge der C++-Programme, die eine Funktion $f : \mathbb{N} \to \{0, 1\}$ berechnen.

Sei $g : \mathcal{P} \to \{0, 1\}$ eine injektive Funktion. Für $P \in \mathcal{P}$ sei f^P die von P berechnete Funktion.

Betrachte eine Funktion $f : \mathbb{N} \to \{0, 1\}$ mit $f(g(P)) := 1 - f^P(g(P))$ für alle $P \in \mathcal{P}$. Eine solche existiert, da g injektiv ist. Offenbar ist $f \neq f^P$ für alle $P \in \mathcal{P}$, und somit wird f von keinem C++ Programm berechnet.

□

Unter den nicht berechenbaren Funktionen geht es nun um das berühmteste Beispiel, dem **Halteproblem**: Es gibt kein Programm, dass für alle Programme und Eingaben entscheiden kann, ob das Programm terminiert.

Wir betrachten die natürlichen Zahlen $\mathbb{N}$ und die Menge aller Programme Q. Da Programme endliche Texte darstellen, können wir alle Programme durchnummerieren. Es gibt also eine surjektive Funktion $g : \mathbb{N} \to Q$ - jede Zahl kodiert ein Programm. Genauso geht die Kodierung auch in die andere Richtung, jedes Programm kodiert eine Zahl: $f : Q \to \mathbb{N}$. Jedem Programm wird eine Zahl zugeordnet.

Wir definieren die Funktion wie folgt:

$$h : \mathbb{N} \times \mathbb{N} \to \{1, 0\} \tag{1}$$

$$h(x, y) = \begin{cases} 1, & \text{wenn Programm g(x) bei Eingabe y hält} \\ 0, & \text{sonst} \end{cases} \tag{2}$$

x: Nummer eines Programms; y: Eingabe; h(x,y) sagt ob Programm stoppt oder nicht.

Diese Funktion heißt **Haltefunktion** und ihr zugehöriges Entscheidungsproblem heißt **Halteproblem**.

Satz 11. *Die Haltefunktion wird von keinem C++ Programm berechnet. Das heißt es existiert kein Programm das h berechnet.*

Proof. Die Beweisidee ist ein Widerspruchsbeweis.

Angenommen, es existiert ein Programm P, das h(x,y) korrekt berechnet. Wir konstruieren ein neues Programm Q, das wie folgt funktioniert:

Bei Eingabe $x \in \mathbb{N}$:

1. Berechne h(x,x)
2. Falls $h(x, x) = 0 \rightarrow$ halte
3. falls $h(x, x) = 1 \rightarrow$ laufe unendlich weiter

Es passiert also genau das Gegenteil von dem, was h voraussagt.

Da Q ein Programm ist und jedes Programm eine Nummer hat, gibt es ein q mit $g(q) = Q$.

Nun betrachten wir h(q,q).

Fall 1: Betrachte h(q,q) = 1. Q(q) hält aber nach Definition von Q läuft es unendlich weiter.

Fall 2: Betrachte h(q,q) = 0. Q(q) läuft unendlich weiter aber nach Definition von Q müsste es halten.

In beiden Fällen entsteht also ein Widerspruch. Also war die Annahme falsch. Die Haltefunktion ist somit nicht berechenbar. □

Man sagt dazu: das Halteproblem ist **nicht entscheidbar**.

6.1 Collatz-Folge

Ein gutes Beispiel für ein Programm, bei dem nicht leicht zu sehen ist, ob es hält, ist die **Collatz-Folge**. Der Algorithmus funktioniert wie folgt: Nach Eingabe von $n \in \mathbb{N}$ wird berechnet, ob die Collatz-Folge von n den Wert 1 erreicht. Dabei geht das Prgraomm wie folgt vor. Wenn n eine gerade Zahl ist, so wird n halbiert. Ansonsten wird es verdreifacht und um 1 erhöht. Dies wiederholt sich bis die Collatz-Folge den Wert 1 erreicht. Bis heute konnte nicht erfasst werden, ob dieser Algortihmus immer, also für jedes n, hält.

Literatur

[1] Stefan Hougardy and Jens Vygen. *Algorithmische Mathematik*. Springer Spektrum, 2. aufl. edition, 2018.